Lingxi V2.0 Technical Report

Trajectory Abstraction for Guidance-Driven Repository-Level Issue Resolution

Version 2.0 — March 2026

This is a brief technical report. Our full paper will be released soon.

1 Introduction

Lingxi is an open-source, multi-agent framework for automating software engineering tasks. In the V1.5 release, we demonstrated that incorporating historical development knowledge and decomposing repair into specialized agents can achieve a **74.60% resolution rate** (373/500) on the SWE-bench-Verified benchmark.

However, V1.5's pattern-oriented historical knowledge has two limitations. First, it compresses history into a small number of high-level takeaways, which can omit stage-dependent details needed during an actual repair run. Second, when the retrieved patterns do not fit the target issue, they may bias the workflow toward incorrect edits.

Lingxi V2.0 addresses these limitations by introducing a *trajectory-to-guidance* mechanism. Instead of extracting static patterns, V2.0 distills **stage-aware procedural guidance** from historical repair trajectories and injects it into the multi-agent workflow. The distilled guidance focuses on *how to localize, validate, and iterate*—the problem-solving procedure—rather than only enumerating bug-type-specific fix patterns. This helps the model make better decisions during the repair process.

On SWE-bench-Verified, Lingxi V2.0 achieves **81.2%** with MiniMax M2.5 (High Reasoning). To the best of our knowledge, this is the **first autonomous agent framework to exceed the 80% threshold** on SWE-bench-Verified.

Table 1: Comparison with state-of-the-art scaffolds on SWE-bench-Verified. (HR = High Reasoning)

Scaffold	**Backbone Model**	**Pass@1 (%)**
Lingxi V1.5	Claude 4 Opus	74.6
mini-SWE-agent v2	Claude 4.5 Opus (HR)	76.8
TRAE	Doubao-Seed-Code	78.8
Sonar Foundation Agent	Claude 4.5 Opus	79.2
live-SWE-agent	Claude 4.5 Opus (Medium)	79.2
Lingxi V2.0 (Ours)	**MiniMax M2.5 (HR)**	**81.2**

2 System Overview

Lingxi V2.0 is built around a standard repository-level repair pipeline with three stages: **Decoder** (localization/diagnosis), **Solution** (plan construction), and **Solver** (patch execution with iterative test-based verification). On top of this pipeline, V2.0 introduces a trajectory-to-guidance mechanism that makes historical repair experience useful during a new repair run. The system follows four phases, as illustrated in Figure 1.

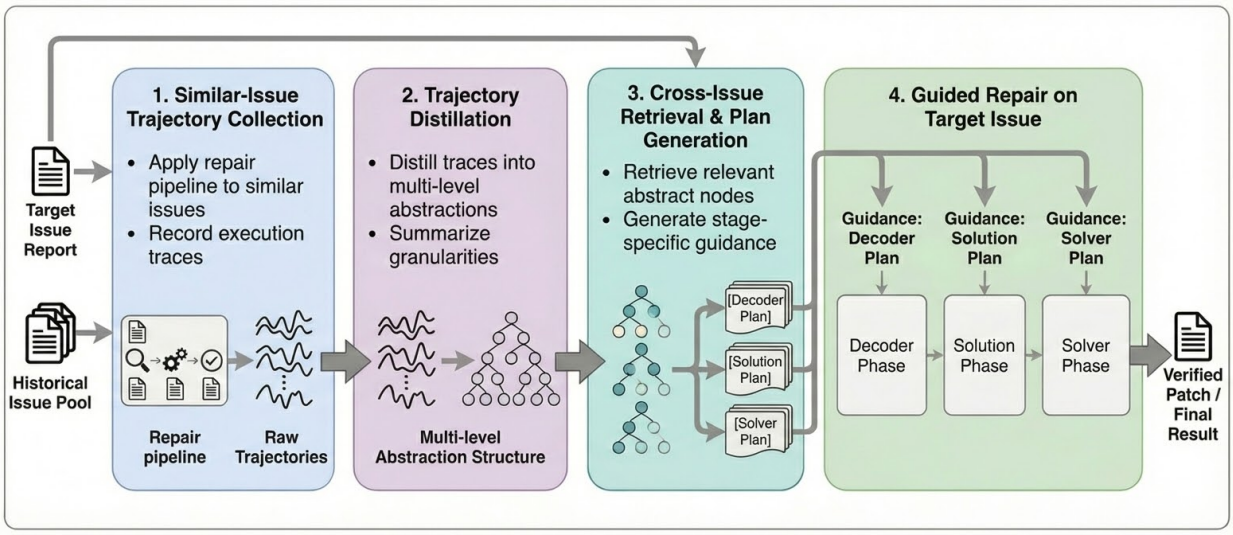

Figure 1: The overall workflow of Lingxi V2.0. The system collects repair trajectories from historical issues, distills them into hierarchical abstractions, retrieves relevant abstract nodes to generate stage-specific plans (Decoder, Solution, and Solver), which guide the final repository-level repair process.

Phase 1 — Similar-Issue Trajectory Collection. Given a target issue report, we identify similar historical issues, execute the Lingxi repair pipeline on each, and record the complete problem-solving trace as a trajectory. Trajectories are stored in a stage-aligned form (Decoder, Solution, Solver) so that downstream components can reuse stage-specific signals without mixing them.

Phase 2 — Hierarchical Trajectory Abstraction. Raw repair trajectories are long and noisy. We distill each stage-aligned trajectory into a hierarchical abstraction tree with multiple levels of granularity: *lower levels* retain concrete sub-goals, *middle levels* capture broader investigation strategies, and *the top level* distills general problem-solving principles. Each node also contains pitfall warnings identified during the original repair.

Phase 3 — Cross-Issue Retrieval & Plan Generation. For a target issue, we retrieve guidance evidence in a stage-aware manner: first via rule-based similarity matching, then through an LLM-based verifier that filters out misleading nodes. Retained nodes are reordered by original execution order, forming coherent guidance. An LLM then generates stage-specific plans for the target issue.

Phase 4 — Guided Repair Execution. We run the Lingxi pipeline on the target issue conditioned on the generated plans. The plans guide but do not fully determine the repair—newly observed evidence may refine the strategy, and failed verification may require revisiting earlier decisions.

3 Key Method: Trajectory-to-Guidance

3.1 Trajectory Collection

For each similar historical issue, we execute the full Lingxi repair pipeline and record the trajectory. Each trajectory is partitioned into stage-specific segments corresponding to the three repair stages (Decoder, Solution, Solver). This stage alignment preserves procedural signals at the stage where they are most useful, allowing later retrieval to target relevant guidance without conflating information from different parts of the repair process.

3.2 Hierarchical Abstraction

We transform each stage trajectory into a multi-level abstraction tree. The design reflects how developers work in practice: they move between high-level strategies and low-level actions, reusing past solutions at the level of detail that fits the current problem.

The abstraction process works bottom-up. Starting from the raw trajectory steps, an LLM groups consecutive steps that share a coherent procedural intent and summarizes each group into a parent node. This grouping-and-summarizing process repeats layer by layer until a small number of root nodes remain. The result is a tree where each level captures repair knowledge at a different granularity:

- **Lower levels** produce concrete sub-goals grounded in diagnostic actions (e.g., "trace how `extra=0` maps to `MAX_NUM_FORMS` in the management form").
- **Middle levels** capture broader investigation strategies (e.g., "trace the end-to-end constraint flow from backend metadata to client-side gating logic").
- **The top level** distills general problem-solving principles (e.g., "presentation-layer configuration $\neq$ server-side capability constraints").

Each node also contains **pitfall warnings**—potential traps identified during the original repair trajectory. These pitfalls are often critical for preventing common mistakes in related issues.

3.3 Retrieval & Plan Generation

Given a target issue, retrieval operates independently for each repair stage through three steps: (1) rule-based scoring matches signals, error types, and keywords from the issue report against node metadata; (2) LLM-based verification evaluates top candidates to filter out nodes that are lexically similar but not applicable; (3) execution-order reordering arranges retained nodes according to their original sequence, forming coherent guidance.

The system then generates a stage-specific **Plan** for each stage by prompting an LLM with the target issue report and the corresponding guidance. During plan generation, the model selectively instantiates applicable procedures and discards irrelevant ones, producing a plan that preserves useful procedures while grounding actions in the current repository.

3.4 Guided Repair

The generated plans are injected into each repair stage as structured guidance. Plans inform the repair process without forcing it to follow a rigid script. After the repair run completes, the resulting trajectory is itself abstracted and incorporated into the procedural memory, enabling the system to continually accumulate knowledge for future issue resolution.

4 End-to-End Example: `django__django-14725`

This section walks through a complete example to show how Lingxi V2.0 turns a seemingly unrelated historical bug into precise architectural guidance for a new issue. Figure 2 illustrates the full process.

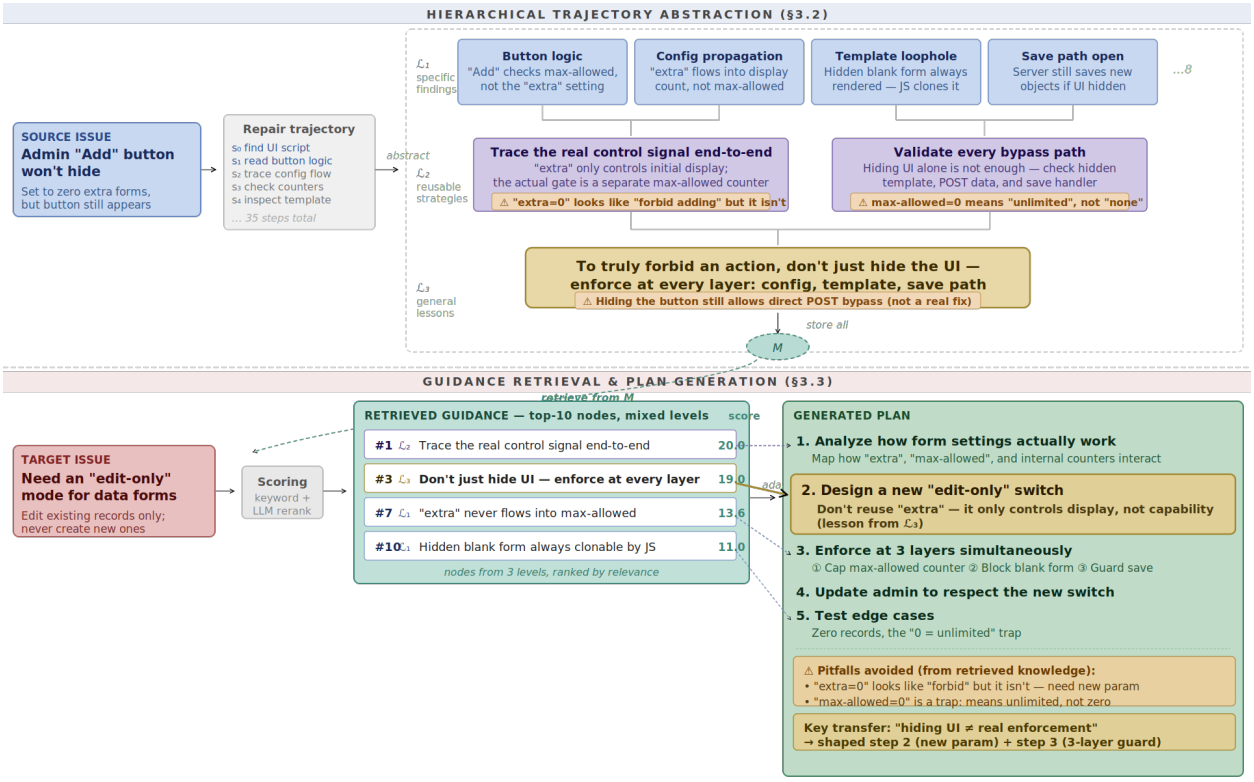

Figure 2: End-to-end example for `django-14725`. **Top**: the historical issue's repair trajectory is abstracted into a 3-level tree (specific findings $\mathcal{L}_1$, reusable strategies $\mathcal{L}_2$, general lessons $\mathcal{L}_3$), each with pitfall warnings. **Bottom**: for the target issue, relevant nodes are retrieved, scored, and adapted into a concrete repair plan.

4.1 The Setup: Two Issues That Look Nothing Alike

Issue A (the target) asks for a proper "edit-only" mode in Django's formset API—one that *truly* prevents creating new records at the server side, not just hides a button. On the surface, this is an API design task.

Issue B (the history) is a much older, much simpler complaint: "I set `extra=0`, but the '+ Add another' button still shows up." On the surface, this is a UI cosmetic bug.

A human skimming these two tickets would likely not connect them. Yet the Lingxi system discovers that they share a deep structural insight—and that insight turns out to be exactly what Issue A needs.

4.2 What the Historical Repair Uncovered

We ran the Lingxi pipeline on Issue B and recorded a 23-step Decoder trajectory and a 35-step Solution trajectory. What started as "why does this button appear?" quickly turned into a tour of Django's formset internals:

1. The "Add another" button is controlled by JavaScript, not the Django template.
2. JavaScript doesn't check `extra` at all—it reads `MAX_NUM_FORMS` from a hidden management form field.
3. Setting `extra=0` has *zero effect* on `MAX_NUM_FORMS`.
4. Here's the trap: `MAX_NUM_FORMS=0` doesn't mean "zero forms allowed." In Django's convention, zero is a **sentinel value meaning unlimited**. So even if you try to set the cap to zero, JavaScript interprets it as "no limit—keep adding."
5. Even with `extra=0`, the hidden `empty-form` template is always rendered in the page, giving JavaScript a ready-made clone source.

The real lesson is not about a button. It's about a fundamental gap in Django's design: **display settings and capability constraints live in completely different mechanisms**, and conflating them creates false security.

4.3 From Bug Fix to Reusable Wisdom

The system abstracts Issue B's 58 raw trajectory steps into a compact three-level tree:

Level 1 — Sub-goals (8 nodes): The concrete investigative steps—locating the JavaScript file, auditing the management form builder, tracing how `InlineModelAdmin` configuration propagates, checking whether `empty-form` renders, verifying save-path behavior.

Level 2 — Strategies (3 nodes): Cross-issue repair patterns distilled from those steps:

- "When UI behavior doesn't match the configuration, trace how the config propagates to the management form's hidden counters, then how client-side code reads those counters. Don't just patch the surface."
- "Distinguish *display parameters* (how many blank forms to show) from *capability constraints* (whether adding is allowed at all). If the user needs a constraint, changing a display parameter won't do."
- "The real 'add-ability' control points are three: the management form counter, the empty-form template, and the save handler. Check all three."

Level 3 — Principle (1 node): The highest-level takeaway:

> *"To truly forbid an action, don't just hide the UI—enforce at every layer: config entry, management counters, template exposure, and save path."*

Every node carries a pitfall warning about the sentinel value: `MAX_NUM_FORMS=0` means "unlimited," not "none." This is the kind of domain-specific trap that an LLM is unlikely to discover on its own from Issue A's description alone.

4.4 Retrieval: Why a "Button Bug" Matches an "API Design" Request

When the system processes Issue A, it retrieves Issue B's abstractions. The match is unexpectedly strong: both revolve around the `extra` parameter's semantics and the question "how do you *actually* prevent adding?"

Strategy and principle nodes score highest (LLM relevance 0.82–0.95), while sub-goal nodes contribute specific implementation angles (0.55–0.68). Nodes from all three levels are selected, giving the plan generator both high-level architectural direction and low-level diagnostic anchors.

4.5 The Generated Plan: More Than the Sum of Its Parts

The system produces a Decoder Plan (how to analyze the problem) and a Solution Plan (how to fix it). The Solution Plan reads like a mini architecture document:

1. Analyze how `extra`, `max_num`, and `MAX_NUM_FORMS` actually interact.
2. **Design a new `edit_only` parameter**—don't reuse `extra`, because it only controls display. (This directly comes from the Level-3 principle.)
3. Enforce at three layers simultaneously: cap the management form counter, block the empty-form template, guard the save handler.
4. Update admin integration to respect the new switch.
5. Test edge cases, especially the "zero = unlimited" sentinel trap.

Here is the most interesting part: **the Plan goes beyond what Issue B ever touched.** Issue B was purely about admin UI—it never examined `save_new_objects()`. But the Plan *extrapolates* from the principle "enforce at every layer" and derives that the save path must also be guarded. This is not copy-paste from history; it is **adaptive reasoning on top of retrieved experience**.

4.6 Execution: A Five-Layer Fix, First Try

Guided by the Plan, the Solver produces a precise, multi-layer patch:

Table 2: Plan requirements and corresponding Solver implementations.

Plan Requirement	Solver Implementation
Introduce an explicit no-create switch	Add `edit_only` parameter to all three factory functions
Limit add-ability at management form layer	Set `MAX_NUM_FORMS = initial_form_count()` when `edit_only=True`
Server does not trust tampered form counts	`total_form_count()` truncates to `initial_form_count()` for bound edit-only formsets
Stop exposing clone templates to frontend	`empty_form` raises `AttributeError` when `edit_only=True`
Prevent new objects from being saved	`save_new_objects()` returns empty list when `edit_only=True`

All 309 existing tests pass. SWE-bench-Verified marks the patch as **resolved**.

4.7 The Counterfactual: What Happens Without the Plan

Without guidance, the most natural repair paths are all shallow: hide the admin button in the template, tweak the JavaScript, or add an admin-specific check. Every one of these "fixes" addresses a symptom instead of the root cause, and none of them work outside the admin interface.

The Plan doesn't make the Solver *smarter*—it changes the Solver's **starting point**. Instead of beginning with "this is an `extra=0` bug" and working up from symptoms, the Solver starts from "this requires a multi-layer API constraint" and works down to implementation. It doesn't need to rediscover that `extra` is just a display parameter (the Plan already says so), or that `MAX_NUM_FORMS=0` is a trap (the pitfall warnings already flag it). All of the Solver's budget goes to writing and testing code—not to architectural exploration that history has already done.

In one sentence: a historical bug about a button that wouldn't hide, after three levels of abstraction, became the architectural blueprint for a server-side edit-only API—and the Solver nailed it on the first try.

References

[1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. R. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

[2] Minimax. MiniMax M2.5, 2026.

[3] X. Yang, J. Zhou, M. Pacheco, W. Zhu, P. He, S. Wang, K. Liu, and R. Pan. Lingxi: Repository-level issue resolution framework enhanced by procedural knowledge guided scaling. *arXiv:2510.11838*, 2025.

[4] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. R. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

[5] Trae Research Team et al. Trae agent: An LLM-based agent for software engineering with test-time scaling. *arXiv:2507.23370*, 2025.

[6] H. Ruan. Introducing Sonar Foundation Agent, 2025.

[7] C. S. Xia, Z. Wang, Y. Yang, Y. Wei, and L. Zhang. Live-SWE-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint*, 2025.